

Take-Home Assignment: Access Control Across Microservices in a Multi-Tenant Architecture

Problem Statement

Design an enterprise-grade Access Control System for a multi-tenant microservices-based SaaS platform.

The platform consists of multiple services such as:

- User Management
- Expense Management
- Payroll
- Reporting
- Workflow
- Notification
- Invoice Management

The system supports multiple organizations (tenants), where each tenant can have:

- Its own users
- Organizational hierarchy
- Roles and permissions
- Policies and access rules

The platform should support:

- Authentication and authorization

- Fine-grained access control
- Tenant isolation
- Cross-service authorization
- Service-to-service communication
- Dynamic role and permission management
- Auditability and security compliance

The system should be scalable enough to support:

- Thousands of tenants
- Millions of users
- High request throughput
- Complex enterprise access patterns

Your task is to design this system end-to-end.

This assignment is intentionally open-ended.

You are expected to explore the problem space, make reasonable assumptions, identify edge cases, and document your decisions and tradeoffs clearly.

What We Expect From You

Please design and document:

- Functional and non-functional requirements
- High-level architecture
- Authentication and authorization flow
- Multi-tenant isolation strategy
- Access control model
- Service-to-service security approach
- APIs and data models
- Scalability and reliability considerations

- Security and compliance considerations
- Operational concerns (monitoring, auditing, debugging, etc.)

You are encouraged to:

- Make assumptions where necessary
- Explore multiple approaches
- Discuss tradeoffs and alternatives
- Think from both product and engineering perspectives

We are interested in:

- Depth of technical understanding
 - Product and domain thinking
 - Security awareness
 - Distributed systems knowledge
 - Design clarity and reasoning
-

Submission Format

Please submit:

- Executable codebase (github link)
- A design document (PDF/Markdown/Slides)
- Architecture diagrams
- Flow diagrams / sequence diagrams where relevant
- API examples
- Schema diagrams

Please ensure:

- Assumptions are clearly documented
- Tradeoffs are properly explained

- Decisions are well justified